Texelator: Repurposing GPU Texture Hardware for Low-Bit Language Model Inference

Hyun Park
Gyeonggibuk Science High School
Republic of Korea

***Abstract*—Autoregressive language-model decoding repeatedly streams weight matrices while performing comparatively little arithmetic. Four-bit quantization reduces this traffic, but conventional kernels reconstruct low-bit weights on programmable GPU resources. We ask whether reconstruction can instead be assigned to hardware that GPUs already use to decode compressed textures. Texelator maps every 16 consecutive weights to one legal signed-BC4 block and consumes the hardware-reconstructed values directly in GEMV. The mapping alone is not sufficient: scalar texture fetches expose request latency and initially lose to software decoding. We make the path competitive by obtaining four useful values per `tex2Dgather()`, issuing independent gathers before consuming either result, and separating accumulation chains. On an RTX 4060, this schedule raises physical weight throughput from 75.9 to 185.4 GB/s and exceeds Marlin's 165.1 GB/s in a matched 24-layer down-projection sweep. Because ideal BC4 arithmetic does not reproduce the GPU exactly, we measure the complete hardware palette and optimize integer endpoints with activation-weighted error. This offline encoder retains the runtime format—4 bits/weight plus one FP32 row scale—and improves Qwen2.5 quality over the evaluated Marlin-compatible INT4 baseline at 0.5B, 1.5B, and 3B. In an audited native-vLLM harness on an RTX 4080 SUPER, Texelator achieves the highest context-512 decode throughput for three Qwen2.5 models and DeepSeek-Coder-1.3B, including 132.4 tokens/s for Qwen2.5-3B versus 103.4 for GPTQ-Marlin. A second-order BC4 compensation experiment reduces calibration reconstruction error but does not generalize to downstream accuracy, illustrating why format-aware systems require both execution and model-level validation. These results establish fixed-function texture reconstruction as a distinct low-bit inference path rather than a graphics-only facility.**

***Index Terms*—large language models, GPU architecture, quantization, texture units, BC4, inference acceleration**

I. Introduction

For each token produced by an autoregressive language model, the GPU applies many linear transformations to an activation containing only one new sequence position. This regime provides little reuse of a weight matrix before the next matrix is needed. The arithmetic capability of contemporary GPUs can therefore grow faster than the useful work available to it: decode latency is often determined by moving weights rather than multiplying them.

Weight-only quantization addresses the traffic directly. GPTQ, AWQ, and related methods store weights at four bits while retaining FP16 activations [1], [2]. Their optimized kernels unpack selectors, apply scales, and perform mixed-precision arithmetic on the programmable path. Marlin shows how careful pipelining and scheduling can make this approach highly efficient [3]. Nevertheless, low-bit reconstruction still occupies resources in the same GPU execution substrate that performs the model's arithmetic.

GPUs contain another reconstruction path. Compressed textures are stored in block formats and expanded on demand by fixed-function texture hardware [4], [5]. This facility is normally treated as part of the graphics interface, not as a numerical primitive for machine learning. Here, we investigate a simple alternative: encode linear weights as genuine BC4 blocks, let the Texture Unit reconstruct them, and consume the returned values directly in a decode GEMV.

The central challenge is not format conversion. A legal BC4 texture can be created with ordinary CUDA APIs, but the first implementation is slower than a matched software decoder. Texture requests have latency, one scalar fetch uses only one value from a spatial neighborhood, and the BC4 interpolation performed by the GPU is not bitwise equivalent to the apparent floating-point formula. Thus, the hardware path becomes useful only when request schedule, block layout, and offline quantization are designed together.

We present Texelator, a complete weight-only inference path built around this observation. Its runtime representation maps 16 consecutive input columns in one output row to an 8-byte signed-BC4 block. One FP32 scale per output row restores the matrix range. No outlier stream, residual correction, or auxiliary dequantization kernel is required. A gather-based CUDA kernel issues independent texture requests before the corresponding fused multiply-adds and keeps their partial sums independent. An offline encoder measures every endpoint pair on the target hardware and chooses endpoints according to activation-weighted error.

Our study makes four contributions:

- We demonstrate that a standard GPU compressed-texture format can represent LLM weights at exactly four bits/weight and can be consumed directly by a numerical CUDA kernel.
- We identify exposed texture latency—not insufficient BC4 decode throughput—as the failure mode of the naive kernel, and show that gather granularity and request-level parallelism turn the path from slower than software decoding into faster than an optimized Marlin reference.
- We introduce a hardware-exact, activation-aware BC4 encoder that changes only offline bit selection. It preserves the runtime byte count and kernel while substantially recovering the quality lost by min/max BC4.

- We evaluate the path from microarchitecture to complete model execution. The experiments include kernel SASS, all-linear sweeps, native-vLLM decode on RTX 4080 SUPER, Qwen2.5 models from 0.5B to 3B, DeepSeek-Coder-1.3B, perplexity, logit fidelity, and six downstream tasks.

The resulting picture is deliberately qualified. TEXELATOR is strongest for batch-one decode, where low arithmetic intensity makes weight movement dominant. Its current evidence is limited to consumer NVIDIA GPUs and modest model sizes. We also find that a more elaborate Hessian compensation can improve calibration reconstruction while reducing downstream accuracy. We therefore select the simpler activation-aware encoder as the final system and treat second-order compensation as a negative generalization result rather than an automatic quality improvement.

II. Background and Motivation

A. Decode as a weight-streaming workload

For a linear layer with weight $W \in \mathbb{R}^{M\times K}$ and one decode activation $x \in \mathbb{R}^K$, the output is

$$y_m = \sum_{k=0}^{K-1} W_{m,k} x_k. \tag{1}$$

At batch one, each weight contributes one multiply-add before the kernel moves to another matrix. A dense FP16 matrix requires approximately $2MK$ bytes, while the useful arithmetic is approximately $2MK$ operations. This low arithmetic intensity explains why weight-only compression can reduce decode latency even when activation and accumulation precision remain high.

Standard W4A16 systems divide weights into groups, store 4-bit codes and scales, and reconstruct numerical operands inside a mixed-precision kernel. GPTQ uses approximate second-order information to select weights [1]; AWQ protects activation-salient weights through per-channel scaling [2]; Marlin supplies a production-style kernel designed to preserve the bandwidth benefit while performing dequantization [3]. These methods establish a strong baseline: the relevant question is not whether texture hardware beats an unoptimized bit-unpack loop, but whether it can compete with this optimized path.

B. BC4 as a scalar 4-bit representation

BC4 is a single-channel block-compressed texture format. One block represents a 4×4 neighborhood—16 scalar texels—in 64 bits [5]. The block contains two signed 8-bit endpoints and sixteen 3-bit selectors:

$$2 \cdot 8 + 16 \cdot 3 = 64 \ \text{ bits}, \quad \frac{64}{16} = 4 \ \text{ bits/value}. \tag{2}$$

The endpoints define an eight-entry palette. Each selector chooses one palette entry for a texel. Unlike ordinary INT4, therefore, the 16 values do not have independent 4-bit codes: they share two endpoints and the interpolation rule. This coupling is the source of both the hardware opportunity and the quantization difficulty.

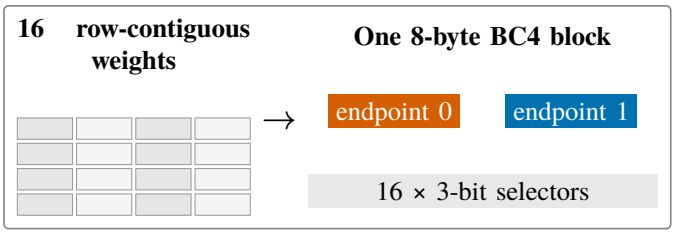

Fig. 1. Row-major mapping used by TEXELATOR. BC4 stores 16 consecutive input columns from one output row; a separate FP32 scale is associated with the row.

C. The fixed-function texture path

CUDA texture objects can reference compressed CUDA arrays. A fetch from such an object returns reconstructed numerical values; software does not explicitly load endpoints, unpack selectors, or interpolate the palette. CUDA additionally provides `tex2Dgather()`, which returns the same component from four neighboring texels in one texture operation [6].

The path does not eliminate memory traffic: compressed blocks and row scales still traverse the memory hierarchy. It changes where reconstruction occurs and how requests enter the GPU. This distinction creates a potential overlap between texture requests and programmable arithmetic, but also introduces a long-latency producer whose result must be scheduled carefully.

III. Texelator Design

A. Weight layout and operator boundary

For each matrix row, TEXELATOR computes one scale

$$\alpha_m = \max_k |W_{m,k}| \tag{3}$$

and normalizes the row before BC4 encoding. Consecutive groups of 16 input columns become BC4 blocks as shown in Fig. 1. The blocks are copied into a CUDA array with a signed BC4 resource view. A runtime handle owns the array, texture object, row scales, and matrix dimensions. The inference operator accepts an FP16 or BF16 activation, accumulates in FP32, and casts directly to the model's output type. Unsupported shapes fall back to the original linear operator.

This layout preserves the reduction order along K: a thread responsible for an output row can traverse consecutive blocks and combine reconstructed weights with contiguous activation values. Padding is unnecessary for the Qwen and DeepSeek matrices evaluated here because their input dimensions are divisible by 16. The storage cost is exactly $M\frac{K}{2}$ BC4 bytes plus $4M$ scale bytes.

B. Gathered decoding and request scheduling

A legal texture mapping alone is insufficient. A scalar `tex2D()` followed by an immediate dependent FMA exposes texture-request latency and uses only one value from the reconstructed neighborhood. TEXELATOR instead maps each lane's logical positions so that one `tex2Dgather()` returns four weights contributing to the same output. A rolling request window issues future gathers before consuming the current

result, and separate accumulators shorten arithmetic dependency chains:

```
float4 w0 = tex2Dgather<float4>(tex, x0,
y, 0);
float4 w1 = tex2Dgather<float4>(tex, x1,
y, 0);

a00 = fmaf(w0.w, x[k+0], a00);
a01 = fmaf(w0.z, x[k+1], a01);
a10 = fmaf(w1.w, x[k+8], a10);
a11 = fmaf(w1.z, x[k+9], a11);
```

The complete kernel consumes all four components from each gather and reduces the partial accumulators only after the loop. The Texture Unit determines the reconstructed values; the CUDA schedule determines whether its latency remains on the critical path.

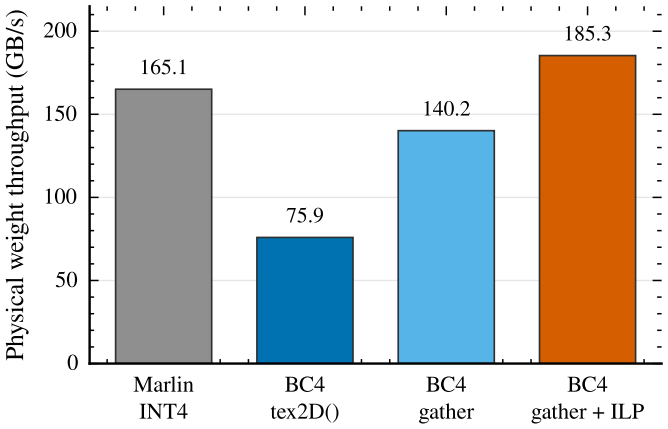

Fig. 2. Physical weight throughput for a rotating sweep of 24 Qwen2.5-0.5B down projections on RTX 4060. Every path processes the same matrices and real decode activations.

As Fig. 2 shows, useful gather granularity raises throughput from 75.9 to 140.2 GB/s; independent issue reaches 185.4 GB/s, versus 165.1 GB/s for Marlin. The final latency is 0.283 ms, a 1.156× speedup over Marlin in this matched 24-layer experiment.

C. Compiled schedule and portable tuning

We inspected the Ada SASS rather than inferring overlap from source order. The prefetch kernel contains adjacent `TLD4` instructions at addresses `0x02c0` and `0x02d0`; the first consuming `FFMA` begins at `0x0470`. The compiler therefore preserves the intended order: issue both gathers, then consume their values. Partial sums reside in distinct registers, including `R23`, `R20`, `R25`, and `R26`. Register use grows from 22 in the one-gather kernel to 39, but ptxas reports zero spill loads, zero spill stores, and no stack frame.

The portable kernel retains the two-request issue-before-consume pattern and exposes a compile-time rolling lookahead K. Here, K counts pending slots: even $K = 1$ issues the next gather before consuming the current result. We freeze the candidate set to $K \in \{1, 2, 3, 4, 6, 8\}$ and select the lowest median all-linear sweep using real calibration activations. All four audited RTX 4080 SUPER workloads select $K = 1$. Every candidate must first agree with the reference kernel within FP16 output tolerance. Autotuning therefore changes only the rolling distance, not encoded weights, arithmetic, or output values. The final batch-one path consumes gathered values directly in FP32 FMA; it does not stage reconstructed weights into an MMA tile.

IV. Hardware-Exact Activation-Aware BC4

The execution path solves only half of the problem. BC4 was designed for spatially correlated texture values, whereas adjacent language-model weights need not vary smoothly. A min/max encoder spreads eight palette entries over the extreme values of each block and can devote too little precision to the region that matters for the layer output.

A. From weight error to output-aware error

Let Q denote the reconstructed matrix. Quantization changes one output by

$$\Delta y_m = \sum_k (W_{m,k} - Q_{m,k}) x_k. \tag{4}$$

We collect 8,192 calibration tokens and estimate the diagonal activation moment

$$h_k = \mathbb{E}[x_k^2]. \tag{5}$$

For a normalized 16-weight block v_i, we choose endpoints and selectors to minimize

$$L(e_0, e_1, s) = \sum_{i=0}^{15} h_i \big(v_i - p_{s_i}(e_0, e_1)\big)^2, \tag{6}$$

where p_s is one of the eight BC4 palette entries. This objective assigns more cost to errors in input channels that commonly carry larger activations. It is not equivalent to AWQ: Texelator does not introduce per-channel runtime scales or protect a separate set of salient weights. The activation statistic is used only to choose the two shared BC4 endpoints and selectors offline.

For fixed selectors, every palette entry is an affine combination $p_{s_i} = a_i e_0 + b_i e_1$. The endpoint update is therefore a weighted least-squares problem:

$$\begin{pmatrix} \sum_i h_i a_i^2 & \sum_i h_i a_i b_i \\ \sum_i h_i a_i b_i & \sum_i h_i b_i^2 \end{pmatrix} \begin{pmatrix} e_0 \\ e_1 \end{pmatrix} = \begin{pmatrix} \sum_i h_i a_i v_i \\ \sum_i h_i b_i v_i \end{pmatrix}. \tag{7}$$

The selectors depend on the endpoints, so this system is not solved once. We initialize with the ordinary block range, assign every weight to its nearest palette entry, solve the 2×2 system for updated endpoints, and reassign selectors. The process repeats until the assignment stabilizes. Because stored endpoints are signed integers, the real-valued solution is rounded and all ± 1 local integer pairs are evaluated with the exact block objective. This alternating procedure is a local discrete optimization, not a claim of a global optimum.

On 28 representative matrices, activation-aware endpoint selection lowers the weighted reconstruction error from 599.191 to 278.529 (–53.5%). Held-out linear output relative RMS falls from 0.1605 to 0.1288, while output cosine rises from 0.98516 to 0.98887.

B. The GPU palette is part of the format

An ideal software decoder computes palette values from the published endpoint interpolation rule. We exhaustively compared that result with the values returned by the RTX 4060 Texture Unit for all $255^2 = 65,025$ ordered endpoint pairs and eight palette entries per pair. Of 520,200 values, 453,378 are not bitwise equal to the CPU floating-point formula. The mean absolute difference is 0.00545 and the maximum is 0.05584 in normalized coordinates.

This discrepancy is too systematic to leave outside the encoder. TEXELATOR records the complete 2,080,800-byte palette for the target GPU and uses those measured values during selector assignment, endpoint search, and quality evaluation. The runtime remains unchanged: the table is an offline encoder asset, not model metadata. We call the combination of measured reconstruction and activation-weighted endpoint optimization the hardware-exact encoder.

The final representation is consequently simple: one ordinary BC4 block per 16 weights and one FP32 scale per output row. Accuracy recovery does not add runtime residuals, outlier indices, secondary weight streams, or correction kernels. Any performance difference between min/max BC4 and the final encoder can only arise from measurement noise or encoded-bit cache effects, not from additional instructions or bytes.

V. EXPERIMENTAL METHODOLOGY

A. Hardware and software

Microarchitectural experiments run on a GeForce RTX 4060 (Ada, compute capability 8.9). They use CUDA events on one stream, 30 warm-up sweeps, 300 measured sweeps, and five independent runs. Each sweep rotates through the 24 Qwen2.5-0.5B down-projection matrices and their corresponding real decode activations, preventing a single matrix from remaining artificially resident. We report the median of run medians unless noted otherwise. Correctness is checked against the established gather kernel; ptxas resource counts and disassembled SASS are retained with the results.

End-to-end experiments run under WSL2 on a GeForce RTX 4080 SUPER (Ada, compute capability 8.9) using CUDA 12.9, PyTorch 2.11, and vLLM 0.23 [7]. We evaluate Qwen2.5-0.5B, 1.5B, and 3B [8] and DeepSeek-Coder-1.3B [9]. Each method uses the same native `LLM.generate` harness, batch size one, retained KV state, prompt hashes, engine arguments, and runtime environment. For every context, the benchmark performs 32 warm-up decode tokens and measures 256 tokens in each of five runs. Reported error bars span the minimum and maximum run result. Before end-to-end timing, Texelator sweeps the frozen lookahead set $K \in \{1, 2, 3, 4, 6, 8\}$ over every encoded linear with representative calibration activations (30 warm-up sweeps, 300 measured sweeps, five runs). All four RTX 4080 workloads select $K = 1$, which still issues one future gather before the current result is consumed.

B. Baselines and audit conditions

The performance comparison includes unquantized FP16, standard symmetric GPTQ-W4A16 group-128, standard AWQ-W4A16 group-128, and TEXELATOR. GPTQ and AWQ are generated with llm-compressor and executed through vLLM's production compressed-tensors Marlin backend. Runtime audits confirm `marlin_gemm` kernel events for both INT4 variants. TEXELATOR installs one fused custom path into the same vLLM engine and replaces all supported transformer linear weights: 168 matrices for Qwen-0.5B, 196 for Qwen-1.5B, and 252 for Qwen-3B. Bias and tensor parallel semantics are retained. Fused-versus-source validation reports maximum absolute error zero for the installation transform.

FP16 is the original unquantized checkpoint, not a quantized surrogate. We do not present a dense reconstruction of INT4 as a performance baseline. This distinction is necessary: a fake-quantized FP16 linear can measure quantization quality, but it does not execute an optimized INT4 kernel.

C. Quality protocol

The final encoder uses WikiText-2 training text [10] and 8,192 calibration tokens. We do not retune calibration size, block order, or endpoint search for larger Qwen models. Logit fidelity is measured on 2,048 held-out teacher-forced tokens using cosine similarity, relative RMS, KL divergence, top-1 agreement, and top-5 agreement against FP16. Perplexity uses 8,192 WikiText-2 test tokens.

Downstream quality follows the Language Model Evaluation Harness [11]. The tasks are HellaSwag [12], ARC-Challenge [13], PIQA [14], WinoGrande [15], BoolQ [16], and MMLU [17]. MMLU is five-shot; the remaining tasks are zero-shot. The six-task score is an unweighted macro average. We keep task accuracy separate from logit reconstruction because a smaller average output error need not preserve the decision boundary of every item.

D. Reproducibility boundaries

All primary 4080 performance rows passed checks for identical engine arguments, runtime environment, prompt hashes, complete raw runs, and expected Marlin or texture kernel events. The RTX 5080 study is not used for a primary claim: long runs exceeded 80 °C and showed 12–39% coefficients of variation, failing a predeclared 3% stability gate. We report this exclusion because selecting the fastest 5080 trial would overstate architecture scaling.

VI. PERFORMANCE EVALUATION

A. From one operator to every linear layer

The kernel result in Fig. 2 is not obtained against a weak software baseline: the final down-projection sweep is 1.156× faster than Marlin. Extending the same operator to every transformer linear yields the results in Table I. On RTX 4060, TEXELATOR reduces the 168-linear Qwen2.5-0.5B sweep from 1.556 to 1.220 ms and the 196-linear 1.5B sweep from 3.995 to 3.538 ms. The relative gain is smaller at 1.5B because projection shapes change the balance among request overhead, useful work, and Marlin's tile efficiency;

nevertheless, the advantage remains after all seven operation families are included.

TABLE I
ROTATING ALL-LINEAR LATENCY ON RTX 4060. EACH SWEEP LAUNCHES EVERY SUPPORTED TRANSFORMER LINEAR ONCE WITH ITS REAL DECODE ACTIVATION.

Model	Linears	Marlin	TEXELATOR	Speedup
Qwen2.5-0.5B	168	1.556 ms	**1.220 ms**	**1.276×**
Qwen2.5-1.5B	196	3.995 ms	**3.538 ms**	**1.129×**

B. *Native-vLLM decode throughput*

Fig. 3 presents the primary end-to-end result. TEXELATOR is the fastest method for every evaluated model at context 512. For Qwen2.5-0.5B and 1.5B, it reaches 183.5 and 163.0 tokens/s, corresponding to 1.178× and 1.183× the FP16 throughput. GPTQ and AWQ are slower than FP16 for these small models in this batch-one harness; this is not treated as a general property of INT4. At small dimensions, quantized-kernel launch, scale handling, and packing overhead can exceed the saved memory time.

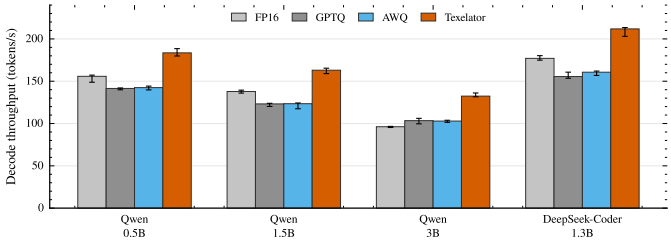

Fig. 3. Batch-one context-512 decode throughput on RTX 4080 SUPER. Bars are medians of five native-vLLM runs; error bars show the run minimum and maximum. GPTQ and AWQ use the production Marlin backend.

The 3B result enters a regime where optimized INT4 already exceeds FP16: GPTQ reaches 103.4 tokens/s and AWQ 102.9, versus 96.2 for FP16. TEXELATOR reaches 132.4 tokens/s—1.281× GPTQ and 1.376× FP16. This comparison is more informative than the small-model ordering because both low-bit paths now overcome their fixed overheads.

DeepSeek-Coder changes the architecture family and matrix shapes without retuning calibration size or encoder search. The same frozen lookahead sweep selects $K = 1$, as it does for all three Qwen models. TEXELATOR reaches 211.8 tokens/s, versus 177.0 for FP16, 160.7 for AWQ, and 155.5 for GPTQ. The 1.197× gain over FP16 does not establish model-family independence, but it shows that the path is not tied to Qwen-specific projection dimensions.

TABLE II
MEDIAN DECODE THROUGHPUT (TOKENS/S) CORRESPONDING TO FIG. 3.

Model	FP16	GPTQ	AWQ	TEXELATOR
Qwen2.5-0.5B	155.8	141.5	142.4	**183.5**
Qwen2.5-1.5B	137.8	123.2	123.4	**163.0**
Qwen2.5-3B	96.2	103.4	102.9	**132.4**
DeepSeek-Coder-1.3B	177.0	155.5	160.7	**211.8**

The five measured contexts reveal whether the context-512 result is selective. Across contexts 128–8192, TEXELATOR remains 1.276–1.319×, 1.303–1.347×, and 1.281–1.342× faster than GPTQ for Qwen-0.5B, 1.5B, and 3B, respectively. DeepSeek-Coder follows a different trend: its ratio falls from 1.352× at context 128 to 1.198× at 4096 and 0.981× at 8192. The long-context loss is consistent with attention becoming a larger fraction of token latency; optimizing linear-weight streaming cannot accelerate that component.

C. *Where the speedup comes from*

The representation does not reduce traffic below four bits per weight. Its advantage over GPTQ/AWQ therefore cannot be attributed to a smaller logical bit width. Instead, the result reflects a different execution allocation: BC4 selector expansion and interpolation are performed by texture hardware, while the CUDA path retains the activation loads, row scale, accumulation, and output store. The kernel progression and SASS together show that this division is useful only when enough independent texture work is visible to the scheduler.

The system is decode-specific. Prefill shapes are routed to the original operator, and TEXELATOR does not claim faster time to first token. At context 512, Texelator TTFT is 116, 402, 826, and 371 ms for Qwen-0.5B, Qwen-1.5B, Qwen-3B, and DeepSeek-1.3B; GPTQ records 13, 17, 32, and 15 ms. We therefore report steady-state decode throughput separately and identify prefill/operator integration as unfinished work. This boundary is important for applications that generate only a few tokens after a long prompt.

VII. MODEL QUALITY AND ENCODER SELECTION

A. *Recovering from naive BC4*

Min/max BC4 confirms that a fast representation is not necessarily a usable one. On Qwen2.5-0.5B, it produces logit cosine 0.9527, relative RMS 0.3013, and perplexity 14.520. The hardware-exact activation-aware encoder improves these values to 0.9816, 0.1889, and 12.346, respectively. Top-1 agreement with FP16 rises from 79.2% to 87.0%. Because both encoders emit the same blocks and row scales, context-512 latency remains within 1% (5.685 versus 5.644 ms/token in the original operator harness).

Table III reports the same fixed encoder across model sizes. The evaluated INT4 quality reference uses symmetric group-128 reconstruction matching the Marlin weights; the 3B row is a dense reconstructed quality measurement and must not be read as kernel latency. In every model, TEXELATOR is closer to FP16 than this reference in cosine, relative RMS, perplexity, and six-task macro accuracy. The gap to FP16 remains visible, particularly for the 0.5B model.

TABLE III
QUALITY SCALING ON QWEN2.5. COSINE AND RRMS COMPARE HELD-OUT LOGITS WITH FP16; PPL USES 8,192 WIKITEXT-2 TEST TOKENS; TASK SCORE IS THE UNWEIGHTED MACRO AVERAGE OF SIX BENCHMARKS.

Size	Method	Cos.↑	RRMS↓	PPL↓	6-task↑
0.5B	FP16	1.0000	0	11.170	53.56%
	INT4 ref.	0.9625	0.2749	13.595	49.94%
	TEXELATOR	**0.9816**	**0.1889**	**12.346**	**51.18%**
1.5B	FP16	1.0000	0	8.242	64.35%
	INT4 ref.	0.9686	0.2450	9.393	62.54%
	TEXELATOR	**0.9832**	**0.1760**	**8.845**	**62.81%**
3B	FP16	1.0000	0	7.2337	68.55%
	INT4 ref.	0.9471	0.3049	8.6817	63.18%
	TEXELATOR	**0.9830**	**0.1785**	**7.7477**	**67.81%**

This comparison does not imply uniform superiority on every task. Quantization can alter discrete answers even when aggregate logit similarity improves. The conclusion supported by Table III is narrower: for the tested Qwen sizes and protocol, the final BC4 encoder retains more FP16 behavior than the evaluated symmetric INT4 quality reference without changing its four-bit runtime path.

B. A second-order compensation that does not generalize

We next adapted GPTQ-style error feedback to BC4. Because 16 weights share endpoints, the quantization unit is a 16-column block rather than one scalar. For block B, remaining columns R, error $E_B = W_B - Q_B$, and inverse damped Hessian $C = H^{-1}$, the offline working weights are updated as

$$W_R \leftarrow W_R - E_B C_{\mathrm{BB}}^{-1} C_{\mathrm{BR}}. \tag{8}$$

The stored representation still contains only the final BC4 endpoints and selectors. Thus, this experiment isolates offline weight choice: it cannot improve or reduce runtime speed.

On 0.5B, compensation appears favorable: cosine increases from 0.9816 to 0.9832, relative RMS falls from 0.1889 to 0.1815, and PPL falls from 12.346 to 12.113. Scaling changes the conclusion. On 1.5B, the six-task average falls from 62.81% to 61.68%. On 3B, logit relative RMS improves from 0.1785 to 0.1722, yet PPL increases from 7.7477 to 7.7698 and all six task accuracies decline; the macro average falls from 67.81% to 65.97%.

This is not explained by failure to optimize the calibration objective. A matrix-level audit on 3B finds that compensation reduces calibration linear output RRMS from 0.02686 to 0.02185 (−18.63%) and the corresponding full output SSE by 33.79%. Paired task predictions move in the opposite direction: 2,194 questions change from correct to incorrect, whereas 1,722 change from incorrect to correct, a net loss of 472 answers.

An error audit explains why the stronger calibration objective is not selected for the final system. Across all 252 Qwen2.5-3B matrices, compensation lowers full calibration output SSE but raises component-wise squared error by 52.19% and diagonal activation-weighted error by 58.25%. Mixed improved and worsened weights occur in 93.07% of BC4 blocks; 63.60% of weights in the top 1% activation-importance percentile worsen. The full Hessian can exploit cross-column cancellation on calibration inputs, whereas shared BC4 endpoints limit where the compensating error is placed.

This evidence supports a calibration-generalization failure for this fixed BC4 experiment, not a general claim against GPTQ. Because the simpler activation-aware encoder is more consistent across model sizes and downstream tasks, it is the encoder used by the final system and every reported Texelator performance result.

VIII. DISCUSSION AND LIMITATIONS

A. What is accelerated?

TEXELATOR does not create extra memory bandwidth and does not reduce the principal weight representation below four bits. It moves a specific operation—selector expansion and endpoint interpolation—from a programmable low-bit kernel to the texture path. The benefit appears when three conditions hold: the workload is weight-streaming, each texture operation returns multiple useful operands, and the kernel exposes a sufficient window of independent requests. Removing any one condition explains a measured failure: prefill is compute-dense, scalar `tex2D()` wastes gather opportunity, and immediate consumption exposes latency.

The experiments suggest that the remaining final-kernel bottleneck is not one mechanism across all shapes. Depth two reaches full calculated occupancy and beats Marlin, while larger static depth can trade occupancy for a slightly lower microbenchmark latency. Small projections are increasingly launch-sensitive; large MLP projections approach physical traffic limits. This shape dependence is why the paper reports both rotating operators and complete-model execution.

B. Portability

BC4 is standardized, but numerical reconstruction and texture throughput are hardware properties. Our encoder explicitly measures the palette because the RTX 4060 does not reproduce an ideal CPU formula bitwise. A deployment on a new architecture should regenerate and hash this 2-MiB table, then validate the operator. The RTX 4080 SUPER shares Ada compute capability 8.9 and produced stable primary measurements. The attempted RTX 5080 experiment did not satisfy the stability gate, so this paper makes no Blackwell speedup claim.

The present kernel also assumes supported dimensions divisible by the BC4 block width and targets NVIDIA's CUDA texture interface. The design principle may transfer to other APIs or compressed formats, but the implementation and exact palette are not vendor-independent.

C. Scope of inference

Our target is batch-one autoregressive decode. Larger batches turn GEMV into GEMM and increase the value of Tensor Cores. The direct FP32-FMA backend selected for batch one therefore does not establish the best design in that regime. Prefill and TTFT are not accelerated. Serving systems should route shapes dynamically or amortize prefill before using TEXELATOR for sufficiently long generation.

Model coverage includes three Qwen sizes and one DeepSeek-Coder model, all at or below 3B parameters. The evidence is sufficient to reject a single-shape artifact, but not to claim broad architecture or scale generalization. Models at 7B and above, mixture-of-experts layers, larger batch sizes, and data-center GPUs remain open evaluations.

D. Quality boundaries

The final encoder is post-training and adds no runtime correction. It remains less accurate than FP16, and WikiText-2 plus six English tasks do not represent code, multilingual, long-form, or safety behavior comprehensively. The GPTQ ablation further shows that optimizing a stronger local reconstruction objective can fail at the model decision level. Future encoders should therefore couple calibration metrics with held-out task checks and should report negative scaling results rather than selecting only the model size where an ablation succeeds.

IX. Related Work

Post-training quantization reduces the storage and bandwidth requirements of LLMs without retraining. LLM.int8() isolates outlier dimensions for mixed 8-bit execution [18], while SmoothQuant migrates activation difficulty into weights to enable W8A8 execution [19]. GPTQ applies approximate second-order information to weight-only quantization [1]. AWQ uses activation statistics to protect salient weights without reconstruction-based fitting [2]. Texelator is closest in spirit to activation-aware weight-only PTQ, but its optimization variables are the shared integer endpoints and selectors of an existing texture format.

Efficient kernels determine whether compressed weights translate into latency reduction. Marlin combines mixed-precision tiling, asynchronous movement, and careful scheduling to approach ideal W4A16 speedups [3]. vLLM integrates optimized model execution with efficient KV-cache management [7]. We use production Marlin inside the same native-vLLM harness as the principal programmable low-bit comparison. The contribution is not a replacement quantization API, but an alternative hardware path for reconstructing the operands consumed by a linear layer.

Compressed texture formats were developed to reduce graphics memory traffic, and GPU texture hardware has long supplied fixed-function sampling and format conversion [4]. BC4/RGTC specifies a single-channel 4×4 block in eight bytes [5], while CUDA exposes texture objects and four-texel gather [6]. Prior systems commonly use textures as caches or read-only access paths. Texelator instead treats a compressed texture format itself as the runtime weight representation and co-designs its offline quantizer with the numerical behavior of the hardware decoder.

X. Conclusion

Texelator demonstrates a four-bit LLM weight path in which standard BC4 blocks are reconstructed by fixed-function texture hardware and consumed directly by a decode GEMV. Four useful values per gather, independent request issue, and separate accumulators raise physical weight throughput to 185.4 GB/s and make the matched 24-layer down-projection sweep 1.156× faster than Marlin on RTX 4060.

The execution path also required a different view of quantization. Ideal BC4 arithmetic did not match the GPU, and min/max endpoints did not preserve LLM outputs. Measuring the hardware palette and fitting shared endpoints to activation-weighted error recovered quality without adding a byte or instruction at runtime. In native vLLM on RTX 4080 SUPER, the resulting system delivers the highest context-512 decode throughput across three Qwen2.5 sizes and DeepSeek-Coder-1.3B, including a 1.281× gain over GPTQ-Marlin for Qwen2.5-3B. The advantage persists across contexts 128–8192 for all three Qwen models; DeepSeek-Coder becomes attention-dominated and reaches parity at context 8192.

The broader conclusion is not that texture hardware universally replaces Tensor-Core quantization. It is that a GPU's graphics-derived fixed functions can become useful ML execution resources when representation, request schedule, and model-level accuracy are designed as one system. The unresolved tasks—stable Blackwell measurement, larger models, prefill integration, and quality beyond the present calibration domain—define the boundary of the current result and the next experiments required to extend it.

References

[1] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=tcbBPnfwxS

[2] J. Lin *et al.*, "AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration," in *Proceedings of Machine Learning and Systems*, 2024, pp. 87–100.

[3] E. Frantar, R. L. Castro, J. Chen, T. Hoefler, and D. Alistarh, "MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models," *arXiv preprint arXiv:2408.11743*, 2024, doi: 10.48550/arXiv.2408.11743.

[4] J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone, and J. C. Phillips, "GPU Computing," *Proceedings of the IEEE*, vol. 96, no. 5, pp. 879–899, 2008, doi: 10.1109/JPROC.2008.917757.

[5] Khronos Group, "Khronos Data Format Specification: RGTC/BC4 Compressed Texture Formats." 2024. [Online]. Available: https://registry.khronos.org/DataFormat/specs/1.4/dataformat.1.4.html

[6] NVIDIA Corporation, "CUDA C++ Programming Guide." 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/

[7] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[8] A. Yang *et al.*, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024, doi: 10.48550/arXiv.2412.15115.

[9] D. Guo *et al.*, "DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence," *arXiv preprint arXiv:2401.14196*, 2024, doi: 10.48550/arXiv.2401.14196.

[10] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel Mixture Models," in *International Conference on Learning Representations*, 2017.

[11] L. Gao *et al.*, "A Framework for Few-Shot Language Model Evaluation." Zenodo, 2024. doi: 10.5281/zenodo.12608602.

[12] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?," in *Annual Meeting of the Association for Computational Linguistics*, 2019.

[13] P. Clark *et al.*, "Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," *arXiv preprint arXiv:1803.05457*, 2018.
[14] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, "PIQA: Reasoning about Physical Commonsense in Natural Language," in *AAAI Conference on Artificial Intelligence*, 2020.
[15] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "WinoGrande: An Adversarial Winograd Schema Challenge at Scale," in *AAAI Conference on Artificial Intelligence*, 2021.
[16] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," in *North American Chapter of the Association for Computational Linguistics*, 2019.
[17] D. Hendrycks *et al.*, "Measuring Massive Multitask Language Understanding," in *International Conference on Learning Representations*, 2021.
[18] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in *Advances in Neural Information Processing Systems*, 2022.
[19] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *International Conference on Machine Learning*, 2023.